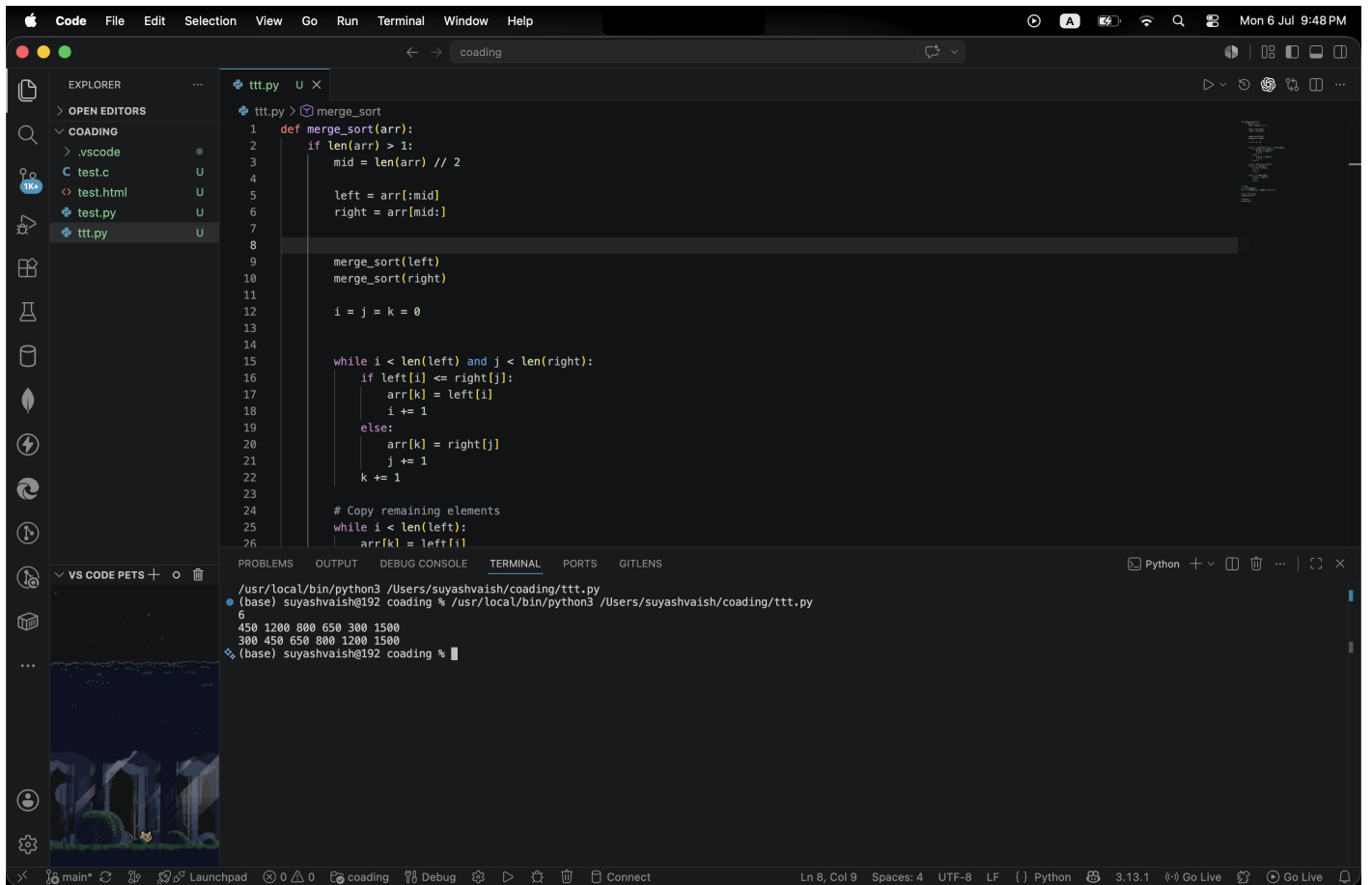


1. Merge Sort

```
def merge(arr, left, mid, right):
    # Create temporary arrays
    L = arr[left:mid + 1]
    R = arr[mid + 1:right + 1]
    i = 0
    j = 0
    k = left
    # Merge the two arrays
    while i < len(L) and j < len(R):
        if L[i] <= R[j]:
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1
        k += 1
    # Copy remaining elements of L
    while i < len(L):
        arr[k] = L[i]
        i += 1
        k += 1
    # Copy remaining elements of R
    while j < len(R):
        arr[k] = R[j]
        j += 1
        k += 1

def merge_sort(arr, left, right):
    if left < right:
        mid = (left + right) // 2
        merge_sort(arr, left, mid)
        merge_sort(arr, mid + 1, right)
        merge(arr, left, mid, right)

# Input
n = int(input())
arr = list(map(int, input().split()))
# Sorting
merge_sort(arr, 0, n - 1)
# Output
print(*arr)
```



2. Merge Sort + Median/Stats

```
def merge(arr, left, mid, right):
    L = arr[left:mid + 1]
    R = arr[mid + 1:right + 1]
    i = 0
    j = 0
    k = left
    while i < len(L) and j < len(R):
        if L[i] <= R[j]:
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1
        k += 1
    while i < len(L):
        arr[k] = L[i]
        i += 1
        k += 1
    while j < len(R):
        arr[k] = R[j]
        j += 1
        k += 1

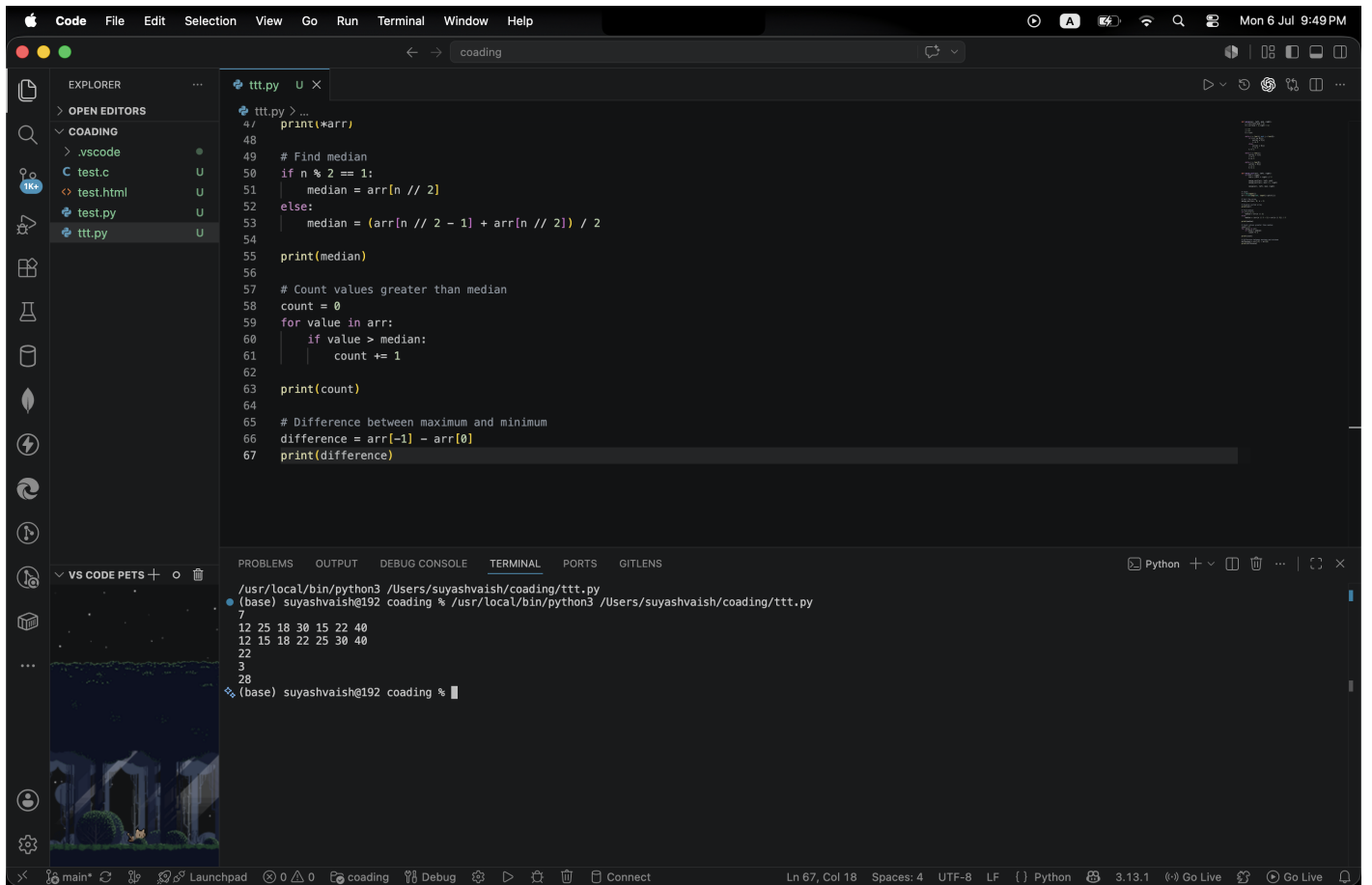
def merge_sort(arr, left, right):
    if left < right:
        mid = (left + right) // 2
        merge_sort(arr, left, mid)
        merge_sort(arr, mid + 1, right)
        merge(arr, left, mid, right)

# Input
n = int(input())
arr = list(map(int, input().split()))
# Sort the array
merge_sort(arr, 0, n - 1)
# Display sorted array
print(*arr)

# Find median
if n % 2 == 1:
    median = arr[n // 2]
else:
    median = (arr[n // 2 - 1] + arr[n // 2]) / 2
print(median)

# Count values greater than median
count = 0
for value in arr:
    if value > median:
        count += 1
print(count)

# Difference between maximum and minimum
difference = arr[-1] - arr[0]
print(difference)
```

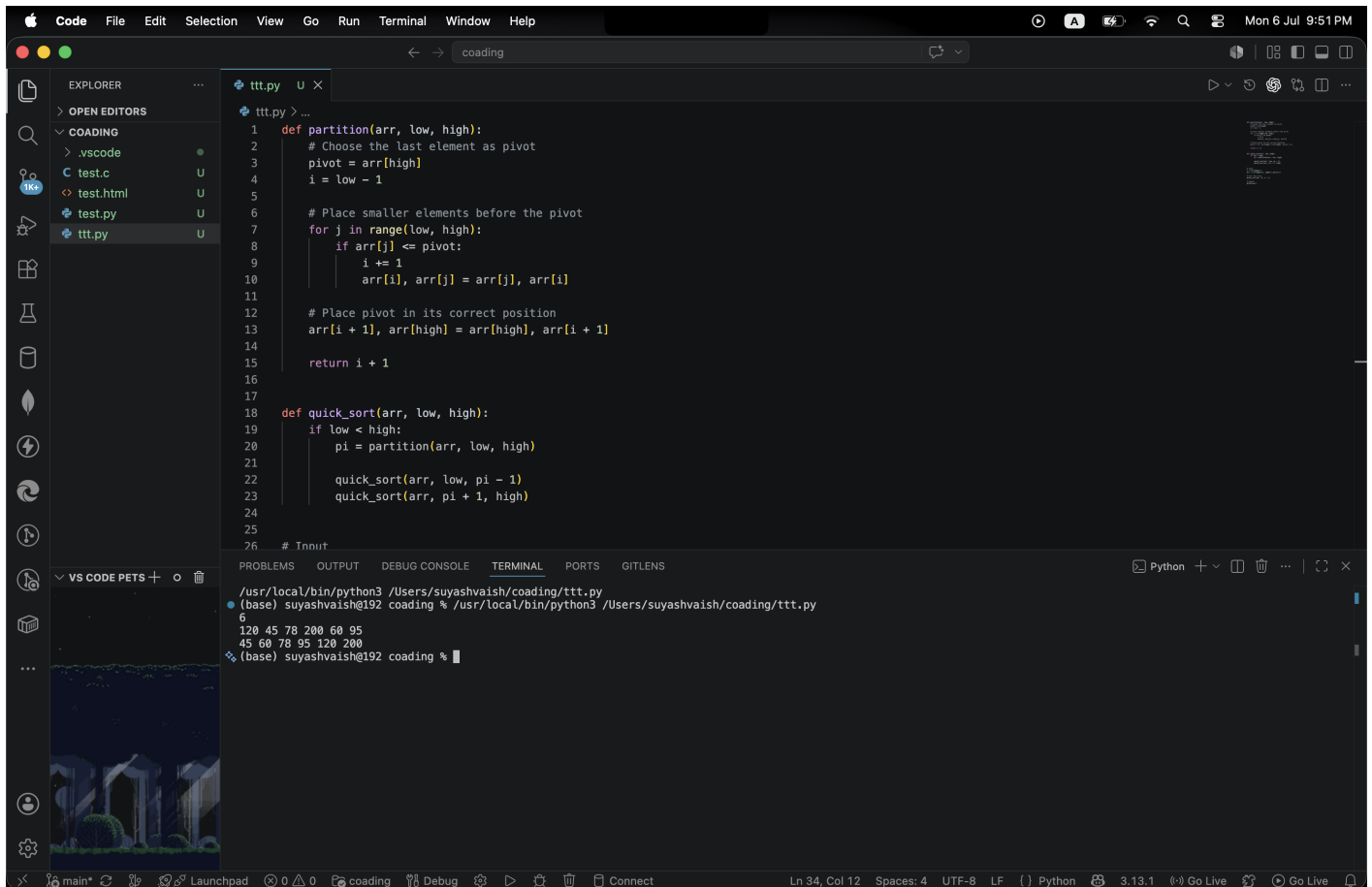


3. Quick Sort

```
def partition(arr, low, high):
    # Choose the last element as pivot
    pivot = arr[high]
    i = low - 1
    # Place smaller elements before the pivot
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    # Place pivot in its correct position
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

# Input
n = int(input())
arr = list(map(int, input().split()))
# Sort the array
quick_sort(arr, 0, n - 1)
# Output
print(*arr)
```



The screenshot shows a Visual Studio Code editor window with a dark theme. The Explorer sidebar on the left shows a project named 'COADING' with files like '.vscode', 'test.c', 'test.html', 'test.py', and 'ttt.py'. The main editor area displays the 'ttt.py' file, which contains the Quick Sort implementation. The code is as follows:

```
1 def partition(arr, low, high):
2     # Choose the last element as pivot
3     pivot = arr[high]
4     i = low - 1
5
6     # Place smaller elements before the pivot
7     for j in range(low, high):
8         if arr[j] <= pivot:
9             i += 1
10            arr[i], arr[j] = arr[j], arr[i]
11
12    # Place pivot in its correct position
13    arr[i + 1], arr[high] = arr[high], arr[i + 1]
14
15    return i + 1
16
17
18 def quick_sort(arr, low, high):
19     if low < high:
20         pi = partition(arr, low, high)
21
22         quick_sort(arr, low, pi - 1)
23         quick_sort(arr, pi + 1, high)
24
25
26 # Input
```

The bottom panel of the editor shows the TERMINAL output. The command executed is `/usr/local/bin/python3 /Users/suyashvaish/coading/ttt.py`, which produces the following output:

```
6
120 45 78 200 60 95
45 60 78 95 120 200
```

The status bar at the bottom indicates the current file is 'Ln 34, Col 12' with 'Spaces: 4', 'UTF-8', 'LF' line endings, and the Python 3.13.1 interpreter is active.

4. Quick Sort Descending + Trade Stats

```
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    # Descending order
    for j in range(low, high):
        if arr[j] >= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

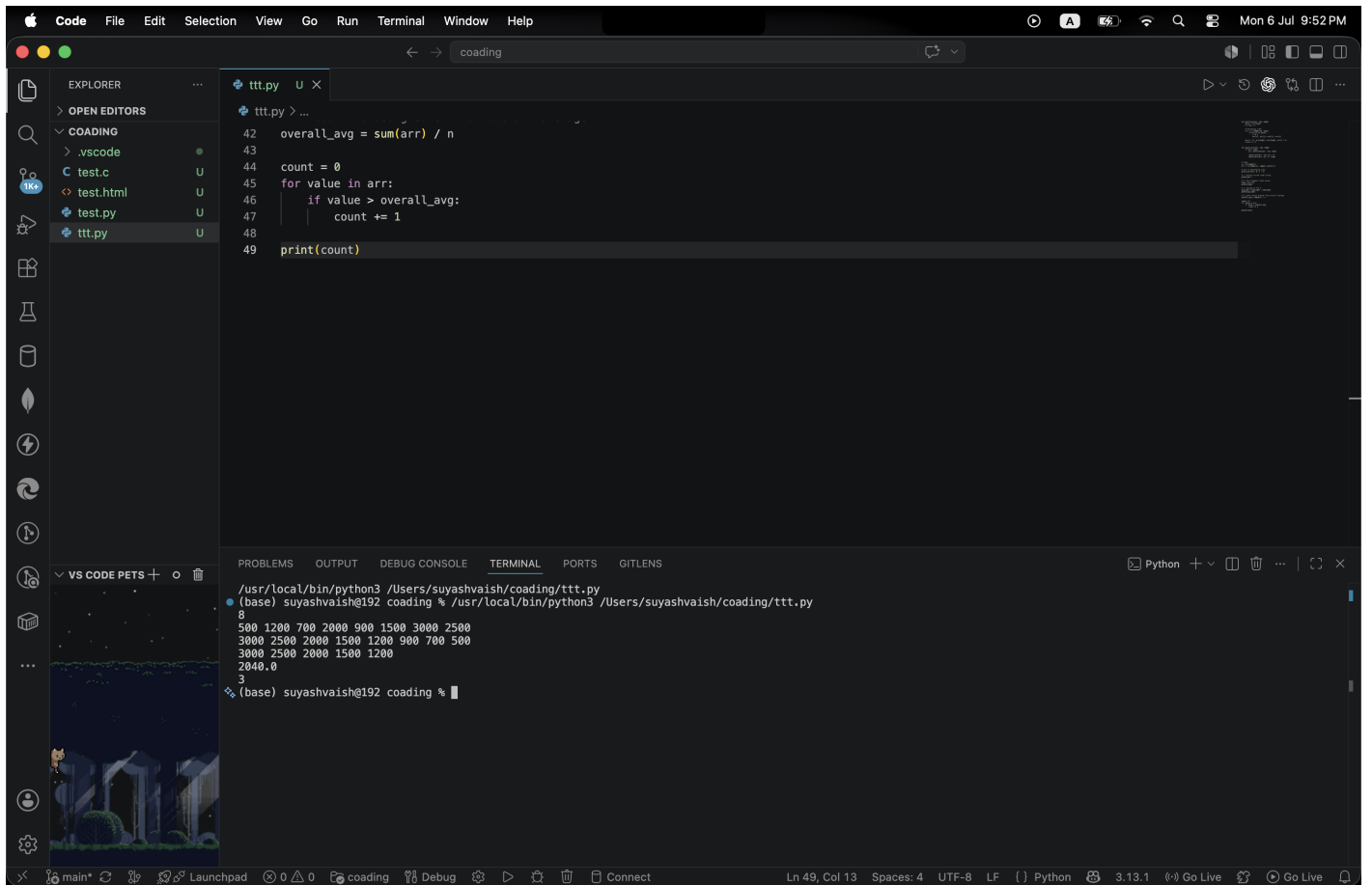
# Input
n = int(input())
arr = list(map(int, input().split()))
# Sort in descending order
quick_sort(arr, 0, n - 1)

# 1. Display sorted trade values
print(*arr)

# 2. Top 5 highest trade values
top5 = arr[:5]
print(*top5)

# 3. Average of Top 5
avg_top5 = sum(top5) / len(top5)
print(avg_top5)

# 4. Count values greater than overall average
overall_avg = sum(arr) / n
count = 0
for value in arr:
    if value > overall_avg:
        count += 1
print(count)
```

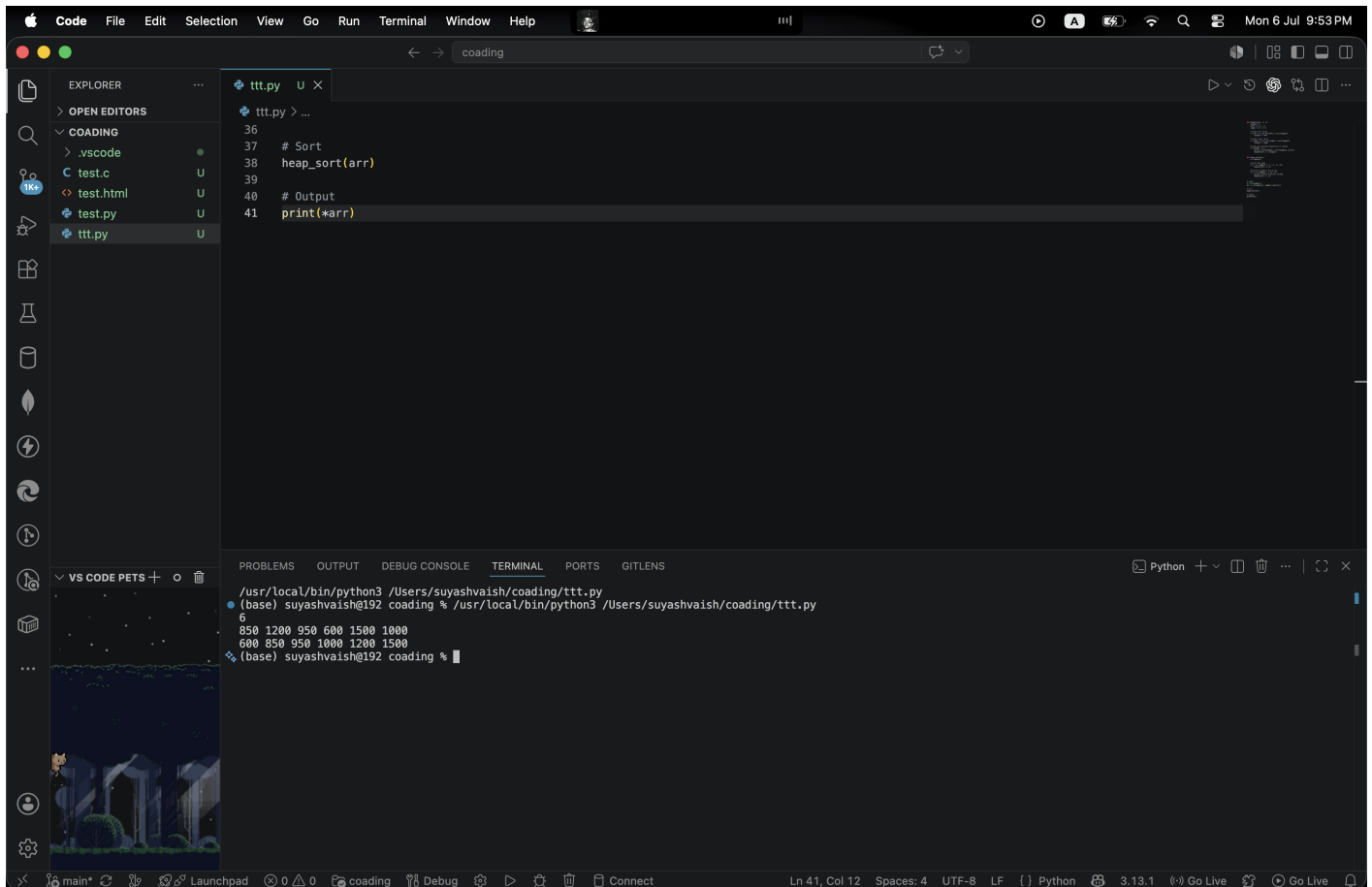


5. Heap Sort

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2
    # Check left child
    if left < n and arr[left] > arr[largest]:
        largest = left
    # Check right child
    if right < n and arr[right] > arr[largest]:
        largest = right
    # Swap and continue heapifying if needed
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)
    # Build Max Heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    # Extract elements one by one
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)

# Input
n = int(input())
arr = list(map(int, input().split()))
# Sort
heap_sort(arr)
# Output
print(*arr)
```



The screenshot shows a Visual Studio Code editor window with the following components:

- Explorer:** Shows the file structure with a folder named "COADING" containing files like .vscode, test.c, test.html, test.py, and ttt.py.
- Editor:** The main area showing the Python code for Heap Sort. The code is in a file named ttt.py. The code is as follows:

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2
    # Check left child
    if left < n and arr[left] > arr[largest]:
        largest = left
    # Check right child
    if right < n and arr[right] > arr[largest]:
        largest = right
    # Swap and continue heapifying if needed
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)
    # Build Max Heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    # Extract elements one by one
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)

# Input
n = int(input())
arr = list(map(int, input().split()))
# Sort
heap_sort(arr)
# Output
print(*arr)
```
- Terminal:** Shows the command to run the script: `/usr/local/bin/python3 /Users/suyashvaish/coading/ttt.py`. The output is: `6 850 1200 950 600 1500 1000 600 850 950 1000 1200 1500`.

6. Heap Sort + Response Time Stats

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2
    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)
    # Build Max Heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    # Extract elements one by one
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)

# Input
n = int(input())
arr = list(map(int, input().split()))
# Sort the array
heap_sort(arr)

# 1. Display sorted response times
print(*arr)

# 2. Fastest response time
fastest = arr[0]
print(fastest)

# 3. Slowest response time
slowest = arr[-1]
print(slowest)

# 4. Average response time
average = sum(arr) / n
print(average)

# 5. Count cases faster than average
count = 0
for time in arr:
    if time < average:
        count += 1
print(count)

# 6. Percentage of cases faster than average
percentage = (count / n) * 100
print(percentage)
```

